

ACTL3142 Tutorial

Week 10: Unsupervised Learning

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T2 2026

Recap: Every Method So Far Has Had a Y

- ▶ **Week 7 (CV, Ridge/Lasso)**: chose λ by minimising held-out prediction error.
- ▶ **Week 8 (Splines, GAMs)**: chose knots and λ by minimising RSS or a CV score.
- ▶ **Week 9 (Trees, Ensembles)**: split the feature space into regions $R_1, \dots, R_J$, choosing each split to reduce RSS or the Gini index, then averaged many trees.

In every case Y did two jobs: it supplied the **loss** that ranked competing models, and it allowed an **honest score** on a held-out test set. This week we observe only $X_1, \dots, X_p$.

What is lost without Y

A regression tree also partitions the data, but every split is justified by the reduction in error on Y . Clustering partitions using the spread of X alone. With no loss to hold out, results cannot be validated against independent data, and the analysis is correspondingly more subjective.

Today: **K-Means & Hierarchical Clustering** and **Principal Component Analysis**.

Two Ways to Simplify Data

Both clustering and PCA simplify the data matrix, but they act on **different sides** of it.

	Clustering	PCA
Acts on	Observations (rows)	Variables (columns)
Finds	Homogeneous subgroups of observations	A low-dimensional representation of the features
Output	A label $1, \dots, K$ for each observation	A few new variables $Z_1, \dots, Z_M$
Criterion	Points in a group are close together	The new variables retain most of the total variance

The two are complementary rather than competing: running PCA first and clustering the resulting scores is standard practice.

Typical applications: market segmentation, fraud detection, grouping patients by condition, and reducing a portfolio to a set of representative policies.

Distance Between Two Observations

Each row of $\mathbf{X}$ is a point in p -dimensional space, $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})^\top \in \mathbb{R}^p$. Clustering requires a rule

$d(\mathbf{x}_i, \mathbf{x}_{i'}) =$ how far apart observations i and i' are,

satisfying $d \geq 0$, $d(\mathbf{x}, \mathbf{x}) = 0$, and $d(\mathbf{x}_i, \mathbf{x}_{i'}) = d(\mathbf{x}_{i'}, \mathbf{x}_i)$.

The default choice is **Euclidean distance**, and both it and its square will be used this week:

$$d(\mathbf{x}_i, \mathbf{x}_{i'}) = \sqrt{\sum_{j=1}^p (x_{ij} - x_{i'j})^2}, \quad \|\mathbf{x}_i - \mathbf{x}_{i'}\|^2 = \sum_{j=1}^p (x_{ij} - x_{i'j})^2.$$

The dissimilarity is a modelling choice, not a technical detail

It is fixed before any algorithm runs, and it defines what “similar” means for the problem at hand. Every method this week inherits that definition, so two analysts using different dissimilarities on identical data can reach different conclusions without either being wrong.

Common Dissimilarity Measures

- ▶ **Euclidean:** $\sqrt{\sum_{j=1}^p (x_{ij} - x_{i'j})^2}$ (the default, and what `kmeans()` assumes)
- ▶ **Manhattan:** $\sum_{j=1}^p |x_{ij} - x_{i'j}|$ (less sensitive to a single discrepant feature)
- ▶ **Simple matching:** $\frac{1}{p} \sum_{j=1}^p I(x_{ij} \neq x_{i'j})$ (all features categorical)
- ▶ **Correlation-based:** $1 - \text{cor}(\mathbf{x}_i, \mathbf{x}_{i'})$ (the shape of a profile, not its level)
- ▶ **Gower's distance:** numerical and categorical features together

Euclidean and correlation distance disagree systematically

A customer who buys a great deal and one who buys very little are far apart in Euclidean distance even when they buy the same kinds of items. Correlation distance treats them as close, since it ignores overall magnitude.

Euclidean and Manhattan distance accumulate per-feature discrepancies, so a feature recorded in larger numbers contributes larger terms.

Standardise unless the features already share units.

K-Means: The Objective

Let $C_1, \dots, C_K$ hold the **indices** of the observations in each cluster, so that every observation belongs to exactly one cluster:

$$C_1 \cup C_2 \cup \dots \cup C_K = \{1, \dots, n\}, \quad C_k \cap C_{k'} = \emptyset \text{ for } k \neq k'.$$

Write $W(C_k)$ for the **within-cluster variation** of cluster k . A good clustering makes the total small:

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K W(C_k), \quad \text{where} \quad W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2.$$

The inner double sum is the **squared Euclidean distance** $\|\mathbf{x}_i - \mathbf{x}_{i'}\|^2$ from the previous section. So $W(C_k)$ is the total squared distance between all pairs of points in cluster k , scaled by $|C_k|$ so that larger clusters are not penalised merely for containing more pairs.

A small $W(C_k)$ means the observations in cluster k lie close to one another. No part of this refers to a response.

K-Means: The Centroid Form

The pairwise definition is not what gets computed. It can be rewritten as

$$W(C_k) = \frac{1}{|C_k|} \sum_i \sum_{j=1}^p (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2,$$

where $\bar{\mathbf{x}}_k = (\bar{x}_{k1}, \dots, \bar{x}_{kp})$ is the **centroid** of cluster k : the mean of each feature over the observations in C_k .

The spread of a group measured pair by pair carries the same information as its spread about its own mean. Discarding the constant 2, K-means solves

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2 = \min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|^2.$$

The centroid form requires $O(nK)$ distance calculations per pass rather than $O(n^2)$. Note that no centroid appears in the definition of $W(C_k)$: it emerges from the algebra, and only because the dissimilarity is a sum of squares.

K-Means: The Algorithm

1. Randomly initialise K centroids.
2. Assign each observation to the cluster whose centroid is **closest** in Euclidean distance.
3. Recompute each of the K centroids as the mean of the observations now assigned to it.
4. Repeat steps 2 and 3 until no observation changes cluster.
5. Repeat steps 1 to 4 many times, retaining the clustering with the smallest total $\sum_k W(C_k)$.

In R: `kmeans(x, centers = K, nstart = 20)`

Local optima

Steps 2 to 4 converge to a **local** minimum of the objective, and which one is reached depends on the random starting centroids. Different seeds therefore give different clusterings. Step 5, supplied by `nstart`, is what makes the output usable.

*K is selected by the **elbow method**: plot total within-cluster variation against K and identify the bend beyond which additional clusters yield little improvement.*

Q2: K-Means Clustering by Hand

[Q2 screenshot placeholder: ISLR2 Q12.3]

Hierarchical Clustering

K-means requires K to be fixed before any structure has been seen.
Hierarchical clustering does not.

Algorithm (agglomerative, or bottom-up):

1. Treat each of the n observations as its own cluster.
2. For $i = n, n - 1, \dots, 2$: compute all pairwise dissimilarities among the i current clusters, merge the least dissimilar pair, and record the height at which they fused.

The merge history is drawn as a **dendrogram**. Cutting it at a given height produces a clustering: a low cut gives many small clusters, a high cut a few large ones. A single run therefore delivers every K from 1 to n .

In R: `hc <- hclust(dist(x), method = "complete")`, then `cutree(hc, k = 3)`.

Similarity is read from fusion height, not horizontal position

Two observations are similar when they fuse **low down** in the tree. Two leaves printed side by side may not join until near the top, making them among the least similar pairs in the data. The branches at any fusion can be swapped left to right without altering a single merge height.

Dissimilarity Between Clusters: Linkage

Once two clusters contain more than one observation, the dissimilarity between clusters requires its own definition. That choice is the **linkage**.

Linkage	Dissimilarity between A, B	Tends to produce
Complete	$\max_{a \in A, b \in B} d(a, b)$	Compact, balanced clusters
Single	$\min_{a \in A, b \in B} d(a, b)$	Extended, “chaining” clusters
Average	$\frac{1}{ A B } \sum_{a \in A} \sum_{b \in B} d(a, b)$	Moderate balance
Centroid	$d(\bar{\mathbf{x}}_A, \bar{\mathbf{x}}_B)$	Can produce inversions

Single linkage chains because one close pair suffices to trigger a merge, so a cluster can absorb points individually and become long and stringy. Complete linkage requires *every* pair to be close and so holds out for tight groups. Average sits between the two, and with complete gives the most balanced trees.

*Centroid linkage can produce an **inversion**, a fusion occurring below the height of one of its own children, which renders the dendrogram difficult to interpret.*

Q1: Dendrograms from a Dissimilarity Matrix

[Q1 screenshot placeholder: ISLR2 Q12.2]

PCA: Why Reduce Dimensions?

When p is large the data cannot be plotted, is slow to model, and is largely redundant. The redundancy arises because features are **correlated**: if one pixel of an image is bright, its neighbour usually is too, so 784 pixel values carry far less than 784 values' worth of information.

What dimension reduction provides:

- ▶ **Visualisation**: plotting the first two or three new variables reveals structure
- ▶ **Speed**: a supervised model on $M \ll p$ features trains considerably faster
- ▶ **Compression**: a compact representation of the data
- ▶ **Uncorrelated inputs**: the new variables are mutually uncorrelated by construction, removing the collinearity problems of Week 3

Central idea: find new axes, the **principal components**, capturing as much variance as possible. PC1 is the direction of greatest variation; PC2 is the direction of greatest remaining variation orthogonal to PC1, and so on.

PCA adopts variance as its measure of information. That is a modelling assumption, and Q3 concerns what it costs.

Q3: Motivations & Drawbacks of Dimensionality Reduction

[Q3 screenshot placeholder: HandsOnML3 Q8.1]

PCA: Setting Up the Problem

We have $\mathbf{X}$, an $n \times p$ data matrix with every column **centred** to mean zero, and no Y . Centring assumes nothing about the data: shifting every observation by the same vector leaves all variances and covariances unchanged, so it merely places the origin at the centroid.

The aim. Replace the p original features with a small number of new variables retaining as much of the variation in the data as possible.

The form of the new variables. Each is a weighted combination of the originals,

$$Z_m = \phi_{1m}X_1 + \phi_{2m}X_2 + \cdots + \phi_{pm}X_p,$$

so constructing a new variable amounts to choosing p weights.

Why variance is maximised. Along a direction where every observation takes a similar value, collapsing the data leaves all n points on top of one another and nothing is retained. A direction with substantial spread keeps the observations distinguishable. Variance therefore serves as the operational measure of information.

PCA: What Exactly Is Being Optimised?

Parameters	The p weights $\phi_m = (\phi_{1m}, \dots, \phi_{pm})^\top$, called the loading vector . These are the only free quantities.
Objective	Maximise $\text{Var}(Z_m)$, the variance of the resulting new variable.
Constraint 1	$\sum_j \phi_{jm}^2 = 1$, so ϕ_m has unit length.
Constraint 2	ϕ_m orthogonal to $\phi_1, \dots, \phi_{m-1}$ (for $m \geq 2$).

The role of each constraint:

- ▶ Without unit length the problem has no solution: doubling every weight doubles each score and quadruples the variance, without limit. Fixing the length ensures only the **direction** is chosen.
- ▶ Without orthogonality every component would return the same answer, since the direction of greatest variance does not change on being asked twice. It forces each component to describe something new.

A loading vector is therefore a direction in $\mathbb{R}^p$, and PCA is a search over directions.

PCA: The First Principal Component

Taking $m = 1$: the **first principal component** is the normalised linear combination of the features with the largest variance,

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p, \quad \sum_{j=1}^p \phi_{j1}^2 = 1,$$

with loadings given by

$$\max_{\phi_{11}, \dots, \phi_{p1}} \underbrace{\frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2}_{\text{Var}(Z_1)} \quad \text{subject to} \quad \sum_{j=1}^p \phi_{j1}^2 = 1.$$

The inner sum is the **score** $z_{i1} = \phi_{11}x_{i1} + \cdots + \phi_{p1}x_{ip}$, the position of observation i on the new axis. Centred columns give $\bar{z}_1 = 0$, so $\frac{1}{n} \sum_i z_{i1}^2$ is exactly the sample variance of the scores.

*Two terms to distinguish: the **loadings** are the p weights defining the axis, one set per component; the **scores** are the n values locating each observation along it.*

PCA: Further Components and the Closest Subspace

The **second** principal component has maximal variance among all normalised linear combinations **uncorrelated** with Z_1 . Because the scores are centred, zero correlation between Z_1 and Z_2 is equivalent to $\phi_2 \perp \phi_1$, so a constraint on the data becomes a geometric constraint on the direction. Continuing yields up to $\min(n - 1, p)$ components.

An equivalent characterisation. Maximising variance along a direction is the same as minimising what remains off it. So ϕ_1 spans the **line closest to the n observations** in average squared Euclidean distance, ϕ_1 and ϕ_2 span the closest **plane**, and the first M loadings span the closest M -dimensional hyperplane.

PCA is not linear regression

Regression singles out Y and minimises **vertical** distances, since only errors in Y count. PCA has no Y , treats all p features symmetrically, and minimises **perpendicular** distances. The objectives differ, so the fitted lines differ even on the same two variables.

The closest-subspace characterisation is what underlies Q4: retaining the first M components gives the best available M -dimensional approximation of the data.

PCA: Interpreting the Loadings

The loading vector identifies which original variables drive a component: a large $|\phi_{jm}|$ means X_j contributes heavily to PC m , a small one means it barely moves along that direction, and variables loading with a common sign move together.

Geometrically, for a correlated cloud of points in two dimensions, PC1 is its long axis and each score is where the perpendicular from a point meets that axis. The number z_{i1} then stands in for the pair (x_{i1}, x_{i2}) , and PC2 is the short axis carrying the remaining spread.

In R, `prcomp(x, scale = TRUE)` returns `$rotation` (loadings), `$x` (scores) and `$sdev` (the standard deviation along each component).

Standardise unless the features share units

PCA operates on raw variances, so the units of measurement determine the answer. In `USArrests`, `Assault` runs into the hundreds while `Murder` is in single digits, and unscaled PC1 is essentially `Assault` alone.

PCA: Proportion of Variance Explained

The **proportion of variance explained** by the m th component is

$$\text{PVE}_m = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = \frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}.$$

With centred columns the numerator is n times the variance of the scores on PC m and the denominator is n times the total variance across all p features, so PVE_m is the share of total variance lying along component m . The PVEs are non-negative, decreasing in m , and sum to 1 over all $\min(n-1, p)$ components.

Selecting the number of components:

- ▶ **Scree plot:** plot PVE_m against m and locate the elbow beyond which the curve flattens.
- ▶ **Cumulative PVE:** the smallest M with $\sum_{m=1}^M \text{PVE}_m \geq 0.90$ or 0.95 .
- ▶ **Downstream performance:** where the components feed a supervised model, treat M as a tuning parameter and select it by cross-validation.

In R: `pve <- pr$sdev^2 / sum(pr$sdev^2); plot(cumsum(pve))`

Q6: PCA on a 1000-Dimensional Dataset

[Q6 screenshot placeholder: HandsOnML3 Q8.5]

Q4: Reversing Dimensionality Reduction

[Q4 screenshot placeholder: HandsOnML3 Q8.3]

Practical Considerations

Unsupervised methods return an answer for any input. Nothing in the output indicates whether that answer is meaningful, so the burden falls on the analysis.

Robustness. There is no correct clustering waiting to be recovered. Vary the dissimilarity, the linkage, K , and the subset of observations used. Findings that persist across these choices are worth reporting; findings that appear under one configuration only are not.

Validation. No independent test set can confirm a clustering or a set of components, because there is no quantity to predict. The available checks are external: do the groups correspond to something recognisable, and do the components admit an interpretation in the language of the problem?

Sensitivity to preprocessing. Standardisation, the choice of dissimilarity, and the treatment of outliers all change the result materially, and none of them is settled by the data alone.

This is the substantive difference from every earlier week. Supervised learning delegates the final judgement to a test set; unsupervised learning cannot, and returns it to the analyst.

Thanks for coming! Best of luck for your exams!

